Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

Аудит безопасности веб-приложения

Дисциплина «Информационная безопасность»

Выполнил:
Студент группы P3431
Нодири Хисравхон

Преподаватель:
Маркина Татьяна Анатольевна

г. Санкт-Петербург
2025г.

Содержание

1 Краткое резюме **3**

2 Подготовка тестового стенда **3**
2.1 Запуск OWASP Juice Shop 3
2.2 Установка и настройка OWASP ZAP 3

3 Автоматизированное сканирование (DAST) **4**
3.1 Процесс сканирования 4
3.2 Результаты сканирования 4

4 Подтверждённые уязвимости **5**
4.1 DOM-based Cross-Site Scripting (DOM XSS) в параметре поиска 5
4.2 SQL Injection (SQLi) в параметре поиска / форме логина 6
4.3 CORS Misconfiguration (Access-Control-Allow-Origin: *) 7
4.4 Передача идентификатора сессии в URL (sid в socket.io) 8
4.5 Open Redirect в эндпоинте /redirect 8
4.6 Sensitive Data Exposure / открытый доступ к /ftp 9

5 Моделирование угроз (Threat Modeling) с помощью STRIDE **10**
5.1 Построение диаграммы потока данных (DFD) 10
5.2 Анализ угроз по STRIDE 10

6 Общие рекомендации по устранению рисков **12**

7 Выводы **12**

1 Краткое резюме

В рамках работы выполнен аудит безопасности тестового веб-приложения *OWASP Juice Shop*, развернутого локально в Docker. Аудит включал:

- DAST (Dynamic Application Security Testing): автоматизированное и ручное подтверждение уязвимостей с помощью OWASP ZAP;
- ручную верификацию отдельных сценариев эксплуатации (PoC);
- моделирование угроз по методике STRIDE на основе диаграммы потоков данных (DFD).

По результатам тестирования выявлен набор уязвимостей, относящихся к категориям OWASP Top 10 (инъекции, ошибки контроля доступа, неверные настройки безопасности). Сформированы рекомендации по устранению рисков и повышению общего уровня защищенности приложения.

2 Подготовка тестового стенда

2.1 Запуск OWASP Juice Shop

Приложение разворачивается через Docker. Пример команд:

```
docker pull bkimminich/juice-shop
docker run --rm -p 127.0.0.1:3000:3000 bkimminich/juice-shop
```

Листинг 1: Запуск OWASP Juice Shop в Docker

После запуска приложение доступно по адресу: `http://localhost:3000/`.

2.2 Установка и настройка OWASP ZAP

Устанавливается OWASP ZAP (Standalone). Далее выполняется настройка браузера/прокси (или использование встроенного браузера ZAP) для перехвата и анализа HTTP(S)-трафика.

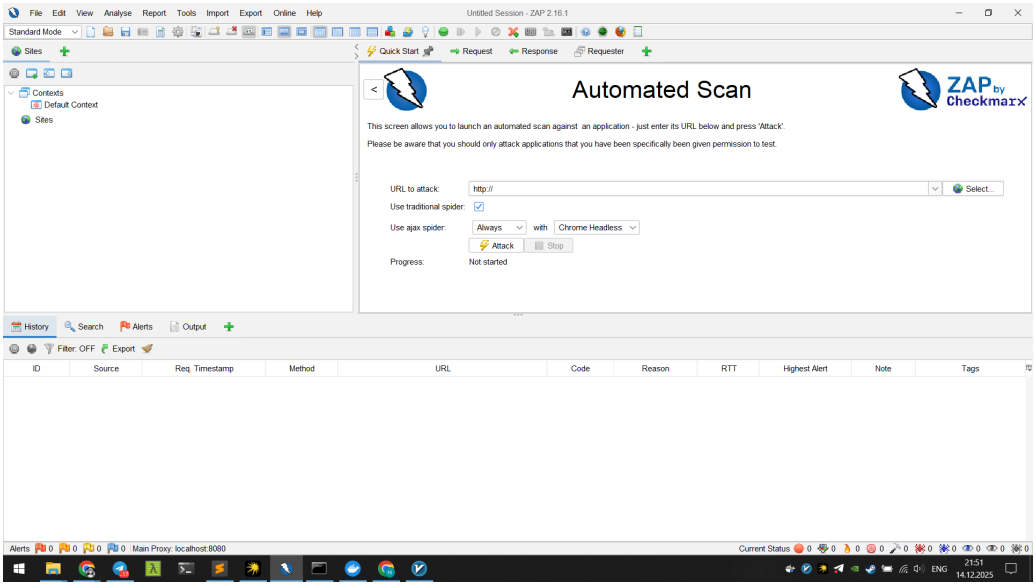


Рис. 1: Настройки прокси/браузера и стартовое окно OWASP ZAP

3 Автоматизированное сканирование (DAST)

3.1 Процесс сканирования

В OWASP ZAP запускается *Automated Scan* для цели `http://localhost:3000/`. Рекомендуется включить:

- Spider (традиционный паук);
- AJAX Spider (для SPA/динамических страниц);
- Active Scan (активная проверка);
- при необходимости — WebSockets scan.

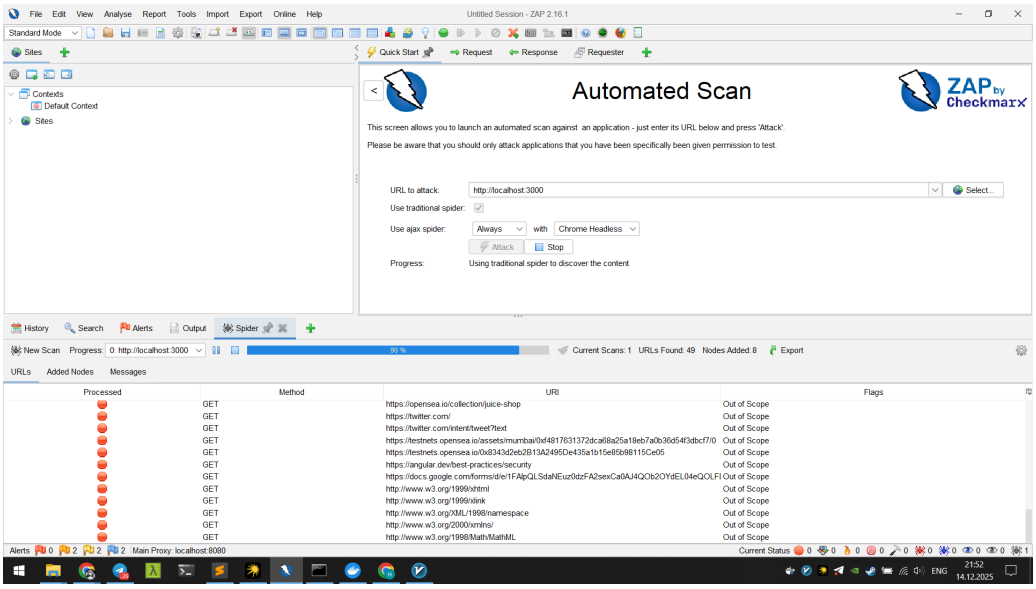


Рис. 2: Окно Automated Scan в OWASP ZAP

3.2 Результаты сканирования

После завершения сканирования анализируется вкладка *Alerts*. Здесь фиксируются найденные потенциальные проблемы с указанием уровня риска (High/Medium/Low/Informational).

Alerts:

- Всего alerts: 35
- High: 1, Medium: 13, Low: 11, Info: 10



Рис. 3: Список Alerts в OWASP ZAP

4 Подтверждённые уязвимости

Ниже приведены уязвимости, подтверждённые в ходе работы. Для каждой уязвимости описаны: краткое объяснение, шаги воспроизведения (PoC), оценка риска (CVSS v4.0) и рекомендации по устранению.

4.1 DOM-based Cross-Site Scripting (DOM XSS) в параметре поиска

Краткое описание. В клиентской части приложения значение параметра поиска некорректно очищается и попадает в DOM, что позволяет выполнить произвольный JavaScript-код в контексте сайта.

Шаги воспроизведения (PoC).

1. Открыть страницу поиска: `http://localhost:3000/#/search?q=<PAYLOAD>`
2. В качестве `PAYLOAD` использовать, например:

```
<iframe src="javascript:alert('xss')">
```

Листинг 2: Пример payload для DOM XSS

Ожидаемый результат: появляется модальное окно `alert()`, что подтверждает выполнение скрипта.

CVSS v4.0: 7.4 High.

OWASP Top 10: A03:2021 – Injection.

Рекомендации.

- Экранировать вывод (использовать безопасные методы работы с DOM: `textContent` вместо `innerHTML`).
- Санитизировать ввод (например, DOMPurify).
- Внедрить CSP (без `unsafe-inline`, с `nonce`/hashes).

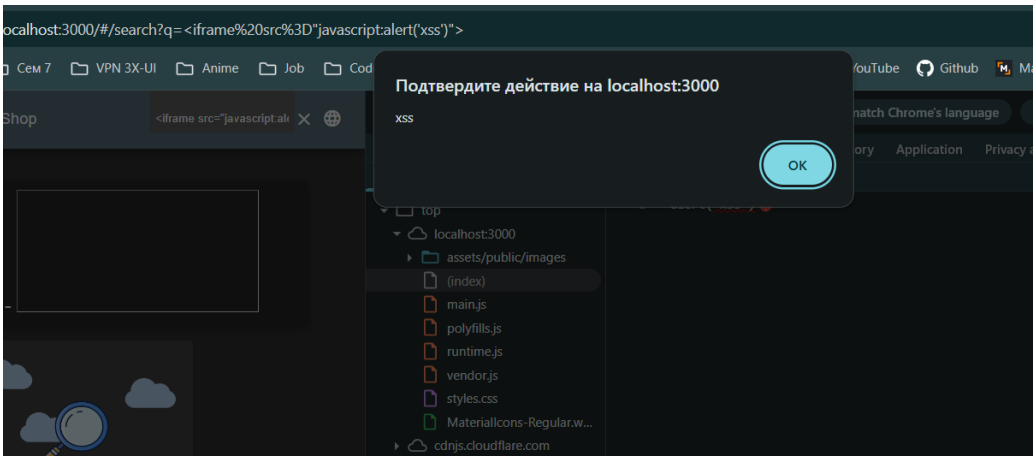


Рис. 4: Скриншот подтверждения XSS (alert)

4.2 SQL Injection (SQLi) в параметре поиска / форме логина

Краткое описание. Пользовательский ввод вставляется в SQL-запрос без параметризации и строгой валидации, что позволяет изменить логику запроса и получить несанкционированный доступ к данным.

Шаги воспроизведения (вариант 1: форма входа).

1. Перейти на страницу `/login`.
2. В поле email ввести:

```
'␣OR␣1=1--
```

Листинг 3: Payload для SQLi в форме логина

3. В поле password указать произвольное значение и нажать *Login*.

Ожидаемый результат: успешная аутентификация (например, под первым пользователем в БД).

CVSS v4.0: 8.8 High.

OWASP Top 10: A03:2021 – Injection.

Рекомендации.

- Использовать parameterized queries / prepared statements / ORM.

- Валидация и ограничение длины ввода (whitelist).
- Принцип наименьших привилегий для пользователя БД.
- Мониторинг и rate limiting на критичных эндпоинтах.

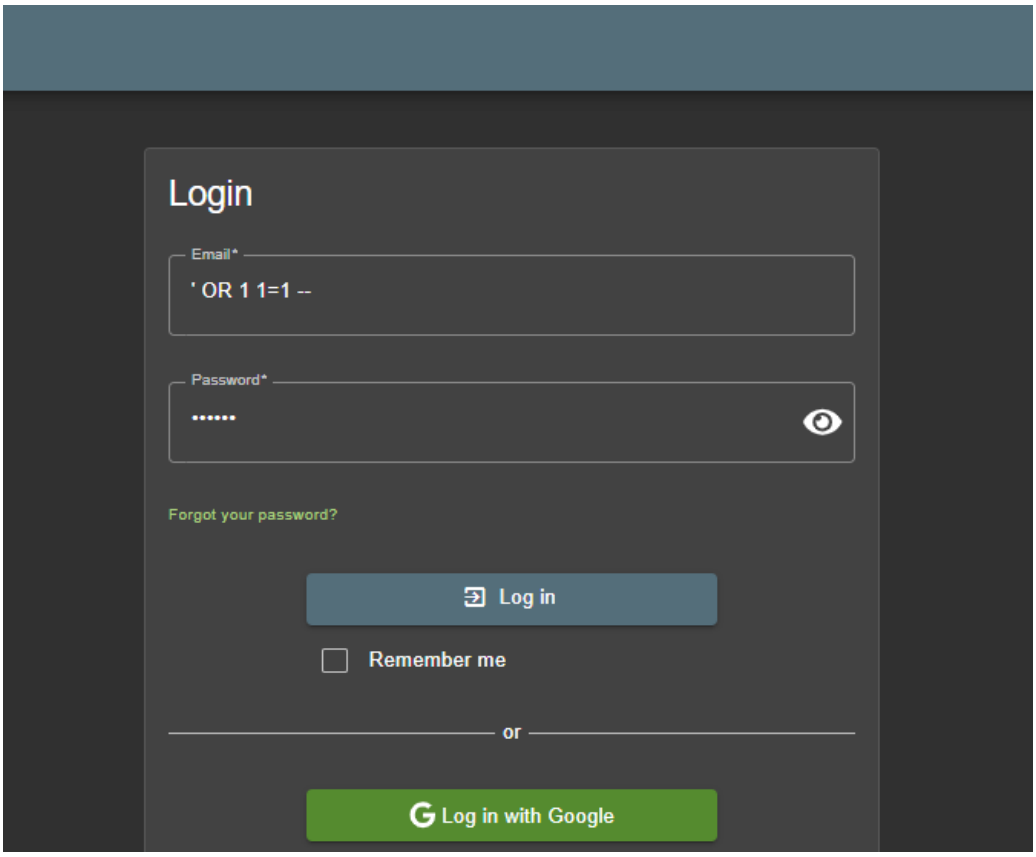


Рис. 5: Подтверждение SQLi

4.3 CORS Misconfiguration (Access-Control-Allow-Origin: *)

Краткое описание. Сервер возвращает `Access-Control-Allow-Origin: *` независимо от `Origin`. Это может позволить стороннему сайту читать ответы API, если есть чувствительные данные и/или неправильно настроена работа с учетными данными.

Подтверждение.

1. Отправить запрос с заголовком `Origin: http://attacker.example`.
2. Убедиться, что в ответе присутствует `Access-Control-Allow-Origin: *`.

CVSS v4.0: 6.5 Medium
OWASP Top 10: A05:2021 – Security Misconfiguration.

Рекомендации.

- Настроить whitelist доверенных Origins.
- Не использовать `*` при работе с cookies/credentials.
- Настроить cookie-флаги `SameSite/HttpOnly/Secure`.



Рис. 6: CORS (Headers/Test Results)

4.4 Передача идентификатора сессии в URL (sid в socket.io)

Краткое описание. Идентификатор сессии (sid) передается/виден в URL или ответах handshake, из-за чего может утечь через логи, историю, referrer или прокси.

Подтверждение.

1. Открыть DevTools → вкладка Network, обновить страницу.
2. Найти запросы к `/socket.io` и убедиться, что sid присутствует в параметрах или теле ответа.

CVSS v4.0: 6.1 Medium.
OWASP Top 10: A02/A05 (Broken Authentication / Security Misconfiguration).

Рекомендации.

- Не передавать критичные токены в URL.
- Использовать HttpOnly+Secure+SameSite cookies или Authorization headers.
- Редактировать/маскировать sid в логах.

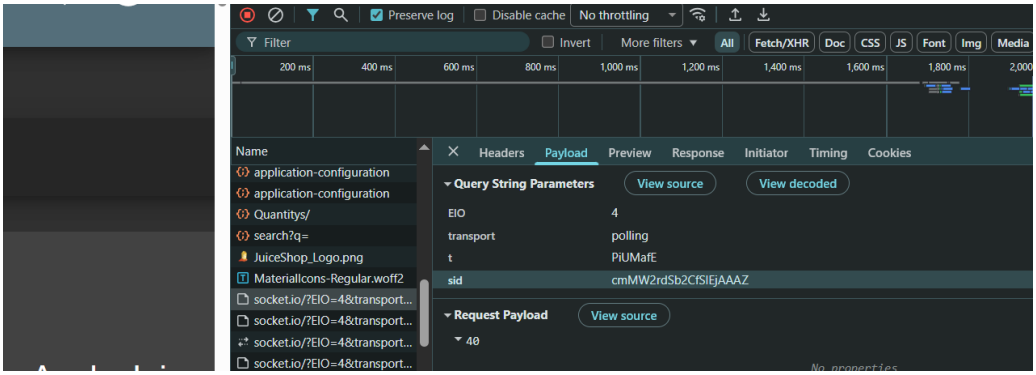


Рис. 7: socket.io с sid

4.5 Open Redirect в эндпоинте /redirect

Краткое описание. Параметр `to` в `/redirect` позволяет перенаправлять пользователя на произвольные домены (в том числе обходя allowlist), что используется для фишинга.

Шаги воспроизведения (пример).

1. Открыть URL вида: `http://localhost:3000/redirect?to=<TARGET>`
2. Подобрать значение `TARGET`, обходя проверки (например, через вложенный URL-параметр).

CVSS v4.0: 4.4 Medium.
OWASP Top 10: A01:2021 – Broken Access Control.

Рекомендации.

- Реализовать строгий белый список разрешенных доменов и схем.
- Нормализовать URL до сравнения (защита от double-encoding и подобных обходов).
- В идеале — не делать серверные редиректы на внешние домены.

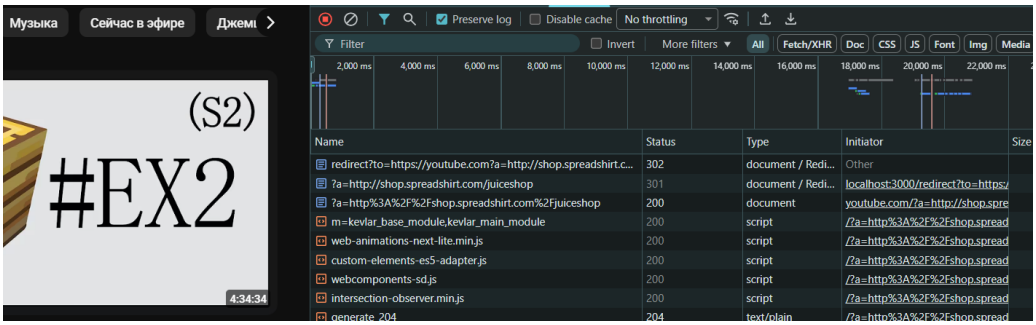


Рис. 8: Редирект на ютуб

4.6 Sensitive Data Exposure / открытый доступ к /ftp

Краткое описание. Эндпоинт `/ftp` (или FTP-сервер) может позволять неавторизованный доступ к конфиденциальным документам, что приводит к утечке информации.

Подтверждение.

1. Открыть `http://localhost:3000/ftp`.
2. Просмотреть список файлов и попытаться скачать документ (например, `acquisitions.md`).

CVSS v4.0: 7.5 High
OWASP Top 10: A01/A05 (Broken Access Control / Security Misconfiguration).

Рекомендации.

- Ограничить доступ (аутентификация/авторизация, закрытие анонимного FTP).
- Применить принцип минимальных привилегий и сегментацию.
- Удалить конфиденциальные файлы из публично доступных директорий.

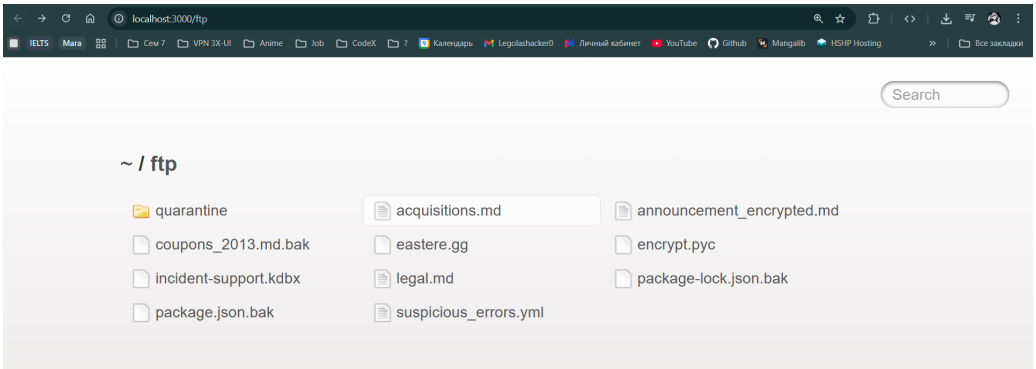


Рис. 9: Листинг /ftp и скачивания файла

5 Моделирование угроз (Threat Modeling) с помощью STRIDE

5.1 Построение диаграммы потока данных (DFD)

Диаграмма потока данных (DFD) описывает ключевые компоненты и потоки:

- Пользователь (Browser) → Веб-приложение (Application);
- Веб-приложение ↔ База данных (Database);
- Веб-приложение ↔ Файловая система/внешнее хранилище (Documents/FTP);
- Потоки: аутентификация, поиск товаров, просмотр товара, корзина, обратная связь, доступ к документам.

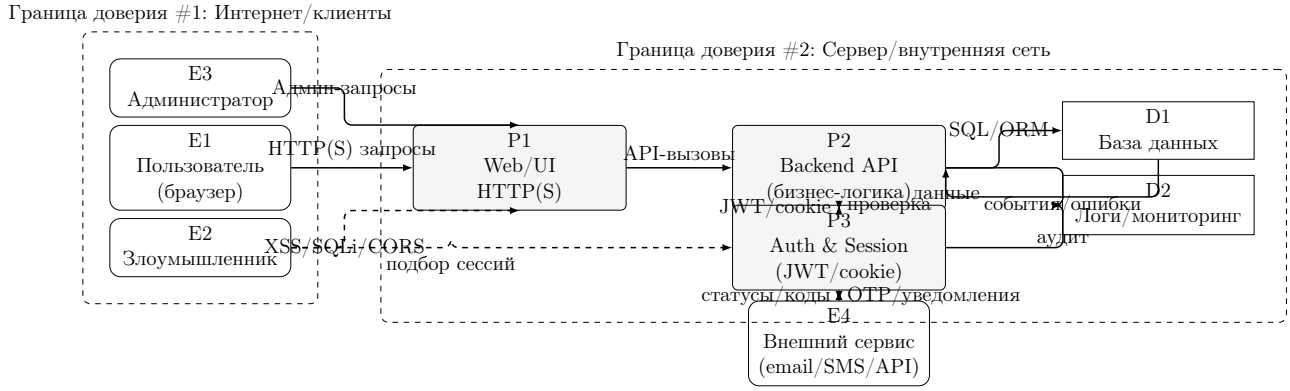


Рис. 10: DFD (Level 1): процессы, хранилища и потоки данных веб-приложения (ортогональная разводка)

5.2 Анализ угроз по STRIDE

Ниже приведены типовые угрозы для выделенных потоков данных с примерами привязки к найденным уязвимостям.

S — Spoofing (Подмена личности)

- Кража сессии/токена через XSS и последующая подмена пользователя.
- Обход аутентификации через SQLi в форме логина.
- Фишинг через Open Redirect (перенаправление на поддельную страницу входа).

T — Tampering (Изменение данных)

- Модификация параметров запросов и влияния на выборку из БД через SQLi.
- Подмена данных на клиентской стороне (например, изменение идентификаторов объектов при слабом серверном контроле).

R — Repudiation (Отказ от совершённых действий)

- Отсутствие полноценного аудита (логирования аутентификаций, покупок, изменений профиля) приводит к невозможности доказать факт действия пользователя.

I — Information Disclosure (Раскрытие информации)

- Утечка документов через `/ftp` или открытый FTP.
- Чтение ответов API через неверный CORS (в зависимости от конфигурации).
- Утечка sid через URL/логи.

D — Denial of Service (Отказ в обслуживании)

- Дорогостоящие SQL-запросы/инъекции могут перегрузить БД (например, payload с задержками или ресурсоёмкими функциями).
- Отсутствие rate limiting на критичных эндпоинтах увеличивает риск DoS/bruteforce.

E — Elevation of Privilege (Повышение привилегий)

- Получение админских привилегий через кражу токена (XSS) или эксплуатацию SQLi.
- Горизонтальная эскалация при недостаточном контроле доступа к объектам (IDOR-сценарии).

Таблица 1: Сводная таблица уязвимостей (по результатам OWASP ZAP)

#	**Уязвимость**	**Риск**	**Доказательства/детали**	**Рекомендации**
1	CORS Misconfiguration	High	Ответ содержит слишком широкую настройку CORS: `Access-Control-Allow-Origin: *` *или* отражение `Origin`. Плейсхолдер: `URL: <endpoint>`, `Origin: https://evil.example`.	Разрешать только origin из белого списка; не использовать `*` для приватных API; не включать `Access-Control-Allow-Credentials: true` без строгого whitelist; ограничить методы/заголовки, настроить корректный preflight.
2	Отсутствует Content-Security-Policy (CSP)	Medium	Заголовок `Content-Security-Policy` отсутствует. Плейсхолдер: `URL: <page/api>`.	Добавить CSP, например: `default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'none'`. Далее уточнить источники под реальные CDN/скрипты.
3	Отсутствует X-Frame-Options / защита от clickjacking	Medium	Заголовок `X-Frame-Options` отсутствует, возможна загрузка сайта во `iframe`. Плейсхолдер: `URL: <page>`.	Добавить `X-Frame-Options: DENY` или `SAMEORIGIN`. Предпочтительно вместе с CSP `frame-ancestors 'none'` / whitelist.
4	Отсутствует X-Content-Type-Options	Low	Заголовок `X-Content-Type-Options: nosniff` отсутствует. Плейсхолдер: `URL: <static/api>`.	Добавить `X-Content-Type-Options: nosniff` для предотвращения MIME-sniffing.
5	Отсутствует Strict-Transport-Security (HSTS)	Medium	При доступе по HTTPS отсутствует `Strict-Transport-Security`. Плейсхолдер: `Host: <domain>`.	Включить HSTS: `Strict-Transport-Security: max-age=31536000; includeSubDomains; preload` (после проверки готовности, что весь сайт реально только по HTTPS).
6	Отсутствует Referrer-Policy	Low	Заголовок `Referrer-Policy` отсутствует, реферер может утекать на внешние домены. Плейсхолдер: `URL: <page>`.	Добавить `Referrer-Policy: no-referrer` или `strict-origin-when-cross-origin` (в зависимости от требований).
7	Cookie без флага HttpOnly	Medium	Сессионная cookie установлена без `HttpOnly`: `Set-Cookie: <name>=...; ...` (нет `HttpOnly`). Плейсхолдер: `URL: <login>`.	Выставить `HttpOnly` для всех cookie с токенами/сессиями, чтобы снизить риск кражи через XSS.
8	Cookie без флага Secure	Medium	Cookie передаётся без `Secure` (может уйти по HTTP при ошибочной конфигурации). Плейсхолдер: `Set-Cookie: <name>=...` (нет `Secure`).	Для HTTPS-сайта выставить `Secure` для всех чувствительных cookie. Принудительно редиректить HTTP→HTTPS.

6 Общие рекомендации по устранению рисков

1. **Защита от инъекций:** параметризованные запросы, валидация входных данных, WAF для критичных эндпоинтов.
2. **Защита от XSS:** контекстное экранирование, санитизация, CSP, запрет inline-скриптов.
3. **Контроль доступа:** серверные проверки прав на каждый запрос, RBAC/ABAC, защита от IDOR.
4. **Безопасная конфигурация:** корректный CORS, security headers, выключить подробные ошибки в проде.
5. **Сессии и cookies:** HttpOnly/Secure/SameSite, не передавать идентификаторы в URL, сокращать TTL токенов.
6. **Защита от DoS:** rate limiting, таймауты запросов, ограничения ресурсоёмких операций, мониторинг.
7. **Непрерывная безопасность:** регулярные SAST/DAST проверки и обновления зависимостей.

7 Выводы

В ходе работы был развернут тестовый стенд OWASP Juice Shop и выполнен аудит безопасности с применением OWASP ZAP (DAST) и STRIDE-моделирования угроз. Подтвержден набор уязвимостей, соответствующих OWASP Top 10, а также предложены практические меры по их устранению и снижению рисков (укрепление контроля доступа, защита от инъекций и XSS, корректная конфигурация CORS и CSP, улучшение управления сессиями, внедрение аудита и мониторинга).